ADR-002 — Node2Vec before Graph Neural Networks

Status: Accepted
Date: 2026-07-06

Context

The recommendation system requires generating vector representations of student nodes so that similarity between students can be computed in an embedding space. Two families of approaches are available: shallow embedding methods such as Node2Vec, and deep learning approaches such as Graph Neural Networks (GCN, GraphSAGE). We need to decide which to prioritize given the project constraints: an 8-week internship, a single developer, and a Must Have MVP to deliver before any research extension.

Decision

We will implement **Node2Vec** as the primary embedding method. Graph Neural Networks (GraphSAGE) will be explored as a bonus component in Sprint 4, only after the MVP is fully delivered.

Reasons

Implementation cost. Node2Vec is available natively in the Neo4j GDS plugin (`gds.node2vec`) and requires no model training infrastructure. A GNN implementation requires setting up PyTorch Geometric, defining a model architecture, writing a training loop with negative sampling, and managing GPU/CPU training — a significantly higher implementation cost for a single developer.

Interpretability. Node2Vec embeddings are learned from random walks biased by two interpretable parameters (`p` for return probability, `q` for exploration). This makes it straightforward to explain what the embedding captures and to tune behavior. GNN representations are harder to interpret and debug, which increases risk in a time-constrained project.

Sufficient baseline quality. For a graph of moderate size (1,000–10,000 nodes), Node2Vec embeddings combined with cosine similarity produce competitive recommendation quality, as shown in the original Grover & Leskovec (2016) paper. There is no evidence that the added complexity of a GNN is necessary to meet the project's KPIs (Precision@10 ⩾ 0.25).

Risk management. Placing GNNs as a conditional bonus in Sprint 4 means that if the MVP takes longer than expected, the GNN component is dropped without any impact on project validation. Reversing this priority would put the entire API and evaluation pipeline at risk.

Consequences

- Node2Vec embeddings are computed via `gds.node2vec.write` and stored as node properties, making them available to the FastAPI recommendation endpoints without recomputation at request time.
- GraphSAGE via PyTorch Geometric will be attempted in Sprint 4 Bloc P2 if Sprint 4 Bloc P1 is completed ahead of schedule.
- The evaluation protocol (edge masking, Precision@10, AUC) is designed to benchmark both Node2Vec and GraphSAGE on identical test sets, enabling a fair comparison in the final report.